



PROCESS SENKRONİZASYONU – SINAV ODAKLI TAM NOT

Kaynak slaytlar: [Ders5_Process_Synchronization_1](#), [Ders6_Process_Synchronization_2](#), [Ders7_Process_Synchronization_3](#) **Hoca:** Dr. Öğr. Üyesi Ertan Bütün – soruları DOĞRUDAN slayttan soruyor!

Bu notta kullanılan işaretler:

- **KIRMIZI / KALIN** = Hoca'nın slaytta kırmızı/vurgulu yazdığı, %90 sorulur
- ☐ **MAVİ terim** = Tanım sorusu gelebilir
- **BOŞLUK DOLDURMA** = En sık gelen soru tipi
- **İŞLEM SORUSU** = Hesap/çalıştırma sorusu gelebilir
- = Sık karıştırılan / tuzak nokta



KONULAR (Slayt başlıkları – sıralama önemli)

1. Process Synchronization
2. The Critical-Section Problem
3. Peterson's Solution
4. Synchronization Hardware
5. Mutex Locks
6. Semaphores
7. Classical Problems of Synchronization
8. Monitors
9. Alternative Approaches



PROCESS SYNCHRONIZATION



Temel Kavramlar

- ☐ **Cooperating process'ler** diğer process'leri etkiler veya onlardan etkilenir; **paylaşılan hafıza (shared memory)** veya **dosya sistemleri** ile veri paylaşırlar.
- **Paylaşılan veriye eşzamanlı erişim** → **tutarsızlık (inconsistency)** problemlerine yol açar.
- Paylaşılan veri üzerinde işlem yapan process'ler arasında veriye erişimin **yönetilmesi** gereklidir.



BOŞLUK DOLDURMA

- Cooperating process'ler veri paylaşımını (paylaşılan hafıza) veya (dosya sistemleri) ile yapar.
- Paylaşılan veriye **eşzamanlı erişim**, (tutarsızlık) problemlerine yol açar.



Banka Hesabı Örneği (Race Condition)

Senaryo	Sonuç
Senkronize / Sıralı Erişim (A 500 çeker → B 1000 yatırır)	✅ Doğru sonuç: 2500 TL
Eşzamanlı Erişim (Race Condition) (ikisi de 2000'i okur)	❌ Hatalı sonuç: 3000 TL

💡 **Açıklama:** İki process aynı anda aynı eski değeri (2000) okuduğu için birinin güncellemesi diğerini **ezer (overwrite)**. Çözüm: **mutex**, **semaphore** vb. senkronizasyon araçları.



Producer-Consumer (Üretici-Tüketici) Problemi

❑ **producer-consumer problemi**, ortak paylaşılan veri olduğunda process'lerin senkronize olması gerektiğine bir örnektir. **shared memory** ile paylaşılan bir **bounded buffer** kullanılarak çözülür.

🔴 İyi bir çözümün sağlaması gereken 3 özellik:

- ❑ **Overflow** (buffer dolduğunda taşma olmamalı)
- ❑ **Underflow** (buffer boşken eleman alınmamalı)
 - ➡ Bu ikisi, buffer eleman sayısını tutan ortak değişken **count** kontrol edilerek sağlanır.
- 🔴 **Mutual exclusion** — buffer'a aynı anda **yalnızca bir** producer VEYA consumer erişebilmeli.



Kod Hatırlatması

```
// PRODUCER                                // CONSUMER
while (true) {                                while (true) {
    /* produce item */                        while (counter == 0)
    while (counter == BUFFER_SIZE)            ; /* do nothing */
        ; /* do nothing */                    next_consumed = buffer[out];
    buffer[in] = next_produced;                out = (out + 1) % BUFFER_SIZE;
    in = (in + 1) % BUFFER_SIZE;                counter--; // ➡ Bir eleman alındı
    counter++; // ➡ Yeni eleman                }
}
```

- 🔴 **counter** değişkeni: buffer'a eleman eklenince **artar**, çıkarılınca **azalır**.

İŞLEM SORUSU: counter++ ve counter-- Race Condition

● counter++ ve counter-- makine düzeyinde **atomik DEĞİLDİR**, 3'er komuta ayrılır:

```
counter++ :                                counter-- :
    register1 = counter                    register2 = counter
    register1 = register1 + 1              register2 = register2 - 1
    counter  = register1                  counter  = register2
```

📊 counter = 5 iken şu sırada çalışırsa:

```
T0: producer  register1 = counter          (register1 = 5)
T1: producer  register1 = register1+1      (register1 = 6)
T2: consumer  register2 = counter          (register2 = 5)
T3: consumer  register2 = register2-1      (register2 = 4)
T4: producer  counter = register1          (counter = 6) ➡
T5: consumer  counter = register2          (counter = 4) ➡
```

- ? Bu sırada count kaç olur? → count = 4 (son yazan T5 kazanır)
- ? T4 ile T5 yer değiştirirse? → count = 6 (son yazan T4 kazanır)
- ✅ Doğru değer 5 olmalıydı! Farklı zamanlarda çalışsalar count = 5 olurdu.

⚠ Bu klasik bir işlem sorusudur — "şu sırada count kaç olur?" diye sorulur.

● RACE CONDITION TANIMI (ezberle!)

❑ **Race condition:** Birden çok process'in aynı verilere **eşzamanlı** erişip değiştirdiği VE sonucun **erişimin gerçekleştiği sıraya bağlı** olduğu durumdur.

- Race condition'a düşmemek için, bir seferde **yalnızca bir** process counter'ı değiştirebilmeli → **process synchronization** gerekir.
- **Multicore** sistemler + **multithreaded** uygulamalarda paralel çalışan thread'ler nedeniyle çok sık görülür.

📝 BOŞLUK DOLDURMA

- Sonucun erişim sırasına bağlı olduğu duruma (race condition) denir.
- Race condition'ı önlemek için process'ler (senkronize) edilmelidir.

2 THE CRITICAL-SECTION PROBLEM

Tanımlar

- Sistemde n tane process var: $\{P_0, P_1, \dots, P_{n-1}\}$
- ☐ **Critical section (kritik bölüm):** Her process'in, **en az bir başka process ile paylaşılan veriye eriştiği ve güncellediği** kod bölümü.
- ☒ **Sistemin önemli özelliği:** Bir process kendi critical section'ında çalışırken **diğer process'lerin hiçbirisi** kendi critical section'ında çalışamaz.
- ☒ **Yani aynı anda iki process critical section'da olmamalıdır.**
- ☐ **Critical-section problemi:** Process'lerin paylaştıkları veriyi senkronize kullanabilmeleri için bir **protokol tasarlama** problemidir.

Process'in Genel Yapısı (P_i)

```
do {  
    [ entry section ]      //  İzin isteme bölümü  
        critical section  
    [ exit section ]      //  Çıkış bölümü  
        remainder section //  Kalan kod  
} while (true);
```

BOŞLUK DOLDURMA (çok sık!)

- İzin için kullanılan kod bölümüne (**entry section**) denir.
- Critical section'dan sonra gelen bölüm (**exit section**).
- Geri kalan koda (**remainder section**) denir.

● CS Çözümünün Sağlaması Gereken 3 GEREKSİNİM (EN ÇOK SORULAN!)

#	Gereksinim	Açıklama
1	☐ Mutual Exclusion (karşılıklı dışlama)	Bir Pİ CS'de çalışıyorsa, diğer hiçbir process CS'de olamaz.
2	☐ Progress (ilerleme)	CS'de kimse yokken, CS'ye girmek isteyen varsa, bir sonraki gireceğın seçimi süresiz ertelenemez .
3	☐ Bounded Waiting (sınırlı bekleme)	Bir process istekte bulunduktan sonra, diğerlerinin CS'ye kaç kez girebileceğine bir limit olmalı.

⚠ **Bu 3 madde sürekli sorulur!** Hem Peterson hem TestAndSet çözümleri bu 3 kritere göre değerlendiriliyor.

◆ Varsayımlar:

- Her process **sıfırdan farklı bir hızda** çalışır.
- Process'lerin **göreceli hızına** dair hiçbir varsayım yapılmaz.

⚙ İşletim Sistemlerinde Critical-Section Yönetimi (2 Yaklaşım)

Yaklaşım	Özellik	Race Condition?
☐ Preemptive kernel	Process kernel modda çalışırken askıya alınabilir (yüksek öncelikli başka process için)	⚠ Race condition OLABİLİR → dikkatli tasarım gerekir
☐ Nonpreemptive kernel	Kernel moddaki process askıya alınamaz (çıkana/bloke olana/CPU'yu bırakana kadar çalışır)	✅ Race condition YOK (aynı anda 1 process aktif)

📝 BOŞLUK DOLDURMA

- Kernel modunda askıya alınmaya izin veren kernel (**preemptive**), izin vermeyen (**nonpreemptive**).
- (Nonpreemptive) kernel'da race condition söz konusu değildir.

3 PETERSON'S SOLUTION

🔑 Temel Özellikler

- **Yazılım tabanlı** critical section çözümüdür.
- **Sadece İKİ process** ile sınırlıdır (Pi ve Pj).
- **2 global (paylaşılan) değişken:**
 - `int turn;` → kimin sırası olduğunu gösterir
 - `Boolean flag[2];` → bir process'in CS'ye girmeye hazır olduğunu gösterir

💻 Algoritma

```
do {  
    flag[i] = true;                // entry section  
    turn = j;  
    while (flag[j] && turn == j);  // 🚦 bekle  
        critical section  
    flag[i] = false;              // exit section  
        remainder section  
} while (true);
```

📝 BOŞLUK DOLDURMA (kritik!)

- `flag[i] = true; turn = j; while(...)` bölümü (**entry section**).
- `flag[i] = false;` bölümü (**exit section**).
- `int turn` ve `Boolean flag[2]` (**global / paylaşılan**) değişkenlerdir.

✅ Peterson 3 Gereksinimi de SAĞLAR

1. **Mutual Exclusion** sağlanır → İki process aynı anda CS'ye giremez (çünkü `turn` aynı anda hem 0 hem 1 olamaz).
2. **Progress** sağlanır.
3. **Bounded Waiting** sağlanır.

? **Örnek soru (slayttan):** "Hiçbir process CS'de değilken P0 ve P1 girmeye çalışsın, **P1 daha önce** `turn` 'ü set ederse hangi process CS'ye girer?" ➡ **CEVAP:** `turn` değişkenini **son yazan kaybeder**. P1 önce `turn=0` yazarsa, sonra P0 `turn=1` yazar → `turn=1` kalır → **P1 CS'ye girer** (çünkü P0 `while(flag[1] && turn==1)` koşulunda takılır).

⚠ EN ÖNEMLİ TUZAK: Peterson Modern Mimaride GEÇERLİ Mİ?

● **HAYIR! Modern bilgisayar mimarilerinde DOĞRU ÇALIŞMASI GARANTİ EDİLMEZ.**

- **Sebebi:** İşlemciler/derleyiciler performans için **komutları yeniden sıralayabilir (reordering)**.
- Peterson'da `flag` ve `turn` atamaları yeniden sıralanırsa → **her iki thread de aynı anda CS'de olabilir!** (Mutual exclusion bozulur)

🔧 **Reordering örneği (slayttaki kod):**

```
boolean flag = false;   int x = 0;           // paylaşılan
// Thread1              // Thread2
while (!flag)           x = 100;
;                       flag = true;
print x;
```

⚠ Reordering yüzünden Thread2'de `flag = true` `x = 100` 'den önce çalışabilir → Thread1, `x` henüz 100 olmadan yazdırabilir → **yanlış sonuç (x=0)**.

● ÇIKARIM:

CS problemine **doğru çözüm** sunmanın tek yolu **uygun senkronizasyon araçları** kullanmaktır:

- **Donanımsal özellikler (hardware)**
- **Üst düzey, yazılım tabanlı API'ler** (kernel ve uygulama programcıları için)

📝 BOŞLUK DOLDURMA

- Peterson çözümü modern mimaride komutların (**yeniden sıralanması / reordering**) nedeniyle doğru çalışmayabilir.

4 SYNCHRONIZATION HARDWARE

Giriş

-  Yazılım tabanlı (Peterson gibi) çözümlerin modern mimaride doğru çalışması **garanti edilmez**.
- CS problemi için donanım ve yazılım çözümleri temelde  **kilitleme (locking)** tabanlıdır.

Single Processor Çözümü

-  Tek işlemcili sistemde basit çözüm: process CS'ye girmeden önce **interrupt'ları disable eder** → **context switch olmaz** → preemption engellenir.
- Bu, **nonpreemptive kernel** yaklaşımıdır.
-  **Multiprocessor için UYGUN DEĞİL!** Tüm işlemcilere mesaj göndermek zaman alır, verimliliği düşürür.

CS Problemini Çözen 3 DONANIM KOMUTU (ezberle!)

1. ☐ **Memory barriers**
2. ☐ **Hardware instructions**
3. ☐ **Atomic variables**

4.1 Memory Barriers

- ☐ **Memory model:** Bilgisayar mimarisinin uygulamalara yaptığı **bellek garantileri**.
- **2 Memory model türü:**

Tür	Açıklama
☐ Strongly ordered	Bir işlemcideki bellek değişikliği diğer tümünce hemen görülür.
☐ Weakly ordered	Bir işlemcideki değişiklik diğerlerince hemen görülmeyebilir .

-  **Memory barrier:** Bellekteki değişikliğin diğer tüm işlemcilere **yayılmasını (görünür kılınmasını) ZORLAYAN** komut.
- Barrier sonrası: tüm `load` / `store` işlemleri tamamlanmadan sonraki işlemler başlamaz.

 **Örnek (memory_barrier ile düzeltme):**


```
// Thread 1           // Thread 2
while (!flag)         x = 100;
    memory_barrier();  memory_barrier();
print x;              flag = true;
```

→ Böylece Thread1'de `flag`, `x`'ten önce yüklenir; Thread2'de `x=100`, `flag=true`'dan önce gerçekleşir. **Sonuç garanti 100.**

- ⚠ Memory barrier'lar **çok düşük seviyeli** işlemlerdir, genellikle **sadece kernel geliştiriciler** kullanır.

📝 BOŞLUK DOLDURMA

- Değişikliğin hemen görüldüğü model (**strongly ordered**), görülmeyebileceği (**weakly ordered**).

⚡ 4.2 Hardware Instructions

- 🔴 Yürütülmesi sırasında **interrupt olmayan, atomik** çalışan özel donanım komutları.
- Multiprocessor'da aynı anda çağrılabilirler bile **sırayla** çalışır, biri bitmeden araya başkası giremez.
- ☐ İki **soyut komut** (gerçek isimleri mimariye göre değişir):
 - `test_and_set()` — bir word içeriğini test edip değiştirir
 - `compare_and_swap()` — iki word içeriğini swap eder

🔒 Locks ile Genel Çözüm

```
do {
    [ acquire lock ]    // ➡ atomik
        critical section
    [ release lock ]
        remainder section
} while (TRUE);
```

- Pi `acquire lock` ile kilidi alırsa CS'ye girer; kilit Pi'deyken Pj giremez.
- 🔴 Atomik olduğu için **belli bir anda sadece bir process kilidi alabilir.**

📖 testandset() Tanımı

```
boolean test_and_set(boolean *target) {
    boolean rv = *target;
    *target = true;           // ← HER ZAMAN true yapar
    return rv;               // ← ESKİ değeri döndürür
}
```

🔴 Mantığı:

- `lock = false` ise → `return false`, `lock = true` yapar (kilidi alır)
- `lock = true` ise → `return true`, `lock = true` kalır (giremez, bekler)

📖 Kullanımı (mutual exclusion):

```
boolean lock = false;    // paylaşılan, başlangıç FALSE
do {
    while (test_and_set(&lock))
        ; /* busy wait */
    /* critical section */
    lock = false;
    /* remainder section */
} while (true);
```

✅ **testandset ile:** Mutual Exclusion ✅ ve Progress ✅ sağlanır. ⚠️ Ama **Bounded Waiting** GARANTİ DEĞİL (basit haliyle) → bunun için ek `waiting[]` dizisiyle çözüm yapılır (aşağıda).

📖 compareandswap() Tanımı

```
int compare_and_swap(int *value, int expected, int new_value) {
    int temp = *value;
    if (*value == expected)
        *value = new_value;    // ← sadece eşitse swap
    return temp;               // ← HER ZAMAN orijinal değeri döndürür
}
```

🔴 Özellikler:

- `test_and_set()` gibi **atomik** çalışır.
- Her zaman `value` 'nun orijinal değerini döndürür.
- `value == expected` olunca `value = new_value` yapılır (swap).

📖 Kullanımı:

```
do {
    while (compare_and_swap(&lock, 0, 1) != 0)
        ; /* do nothing */
        /* critical section */
    lock = 0;
        /* remainder section */
} while (true);
```

BOŞLUK DOLDURMA

- `test_and_set` her zaman target'ı (**true**) yapar ve (**eski/orijinal**) değeri döndürür.
- `compare_and_swap` her zaman (**orijinal/value**) değerini döndürür.

testandset ile Bounded-Waiting Çözümü

```
do {
    waiting[i] = true;
    key = true;
    while (waiting[i] && key)
        key = test_and_set(&lock);
    waiting[i] = false;
        /* critical section */
    j = (i + 1) % n;
    while ((j != i) && !waiting[j])
        j = (j + 1) % n;
    if (j == i)
        lock = false;          //  kimse beklemiyorsa kilidi bırak
    else
        waiting[j] = false;    //  sıradakine ver
        /* remainder section */
} while (true);
```

-  **TestAndSet mantığı:**
 - `if lock=false → return false, lock=true`
 - `if lock=true → return true, lock=true`
- Bu çözüm sayesinde sıradaki bekleyen process'e sıra geçer → **Bounded Waiting sağlanır.**

12
34

4.3 Atomic Variables

-  `compare_and_swap()` genellikle **doğrudan mutual exclusion için kullanılmaz**; CS problemini çözen **diğer araçları oluşturmak için temel yapı taşıdır**.
- ☐ **Atomic variables:** `integer`, `boolean` gibi temel veri türleri üzerinde **atomik işlemler** sağlar.
-  **Tek bir değişken** güncellenirken mutual exclusion sağlamak için kullanılır.
- Genellikle `compare_and_swap()` ile uygulanır.

Örnek: atomik increment

```
void increment(atomic_int *v) {  
    int temp;  
    do {  
        temp = *v;  
    } while (temp != compare_and_swap(v, temp, temp+1));  
}  
// kullanım: increment(&sequence);
```

ÖNEMLİ TUZAK:

 **Atomic variables her koşulda race condition'ı TAM ÇÖZEMEZ!** Örn. bounded buffer'da `count` atomik olsa bile: buffer boşken 2 consumer beklerse, 1 producer eleman ekleyince **ikisi de while'dan çıkar**, `count=1` olmasına rağmen **ikisi de tüketmeye çalışır**.

BOŞLUK DOLDURMA

- Atomic variables (**tek bir**) değişken güncellenirken mutual exclusion sağlar ama her koşulda race condition'ı (**tamamen çözemez**).

5

MUTEX LOCKS

Temel

-  Donanım çözümleri **karmaşıktır ve uygulama geliştiriciler erişemez** → OS tasarımcıları **yüksek seviye yazılım araçları** geliştirdi.
- ☐ **En basit araç: mutex (MUTual EXclusion) lock.**
- Process CS'ye girmeden  `acquire()` ile kilidi alır, çıkınca  `release()` ile bırakır.
- Kilit durumu için **boolean** `available` değişkeni kullanılır.

Kod

```
acquire() {
    while (!available)
        ; /* busy wait */
    available = false;
}
release() {
    available = true;
}
// kullanım:
do {
    [ acquire lock ]
        critical section
    [ release lock ]
        remainder section
} while (true);
```

-  `acquire()` ve `release()` **atomik** yürütülmeli (genellikle atomik donanım komutlarıyla).

DEZAVANTAJ ve Spinlock

-  **Temel dezavantaj: busy waiting (meşgul bekleme)** gerektirir → bekleyen process CPU'yu boşuna meşgul eder.
- ☐ Mutex lock'a **spinlock** da denir (process kilidi beklerken "**spin**" / **döner**).
-  **Spinlock avantajı: context switch'e gerek yok** (context switch zaman alır).

Sistem	Spinlock uygun mu?
Tek çekirdekli (multiprogramming)	 Uygun DEĞİL (bekleyen CPU'yu işgal eder, hiçbir iş yapılmaz)
Çok çekirdekli (multicore), kilit kısa süre	 TERCİH EDİLİR (bir thread spin ederken başka çekirdekte diğeri çalışır)

BOŞLUK DOLDURMA

- Mutex lock'a (**spinlock**) da denir.
- Mutex lock'un temel dezavantajı (**busy waiting** / **meşgul bekleme**).
- Kilit alma fonksiyonu (**acquire()**), bırakma (**release()**).
- Mutex lock durumu (**available**) adlı boolean değişkenle tutulur.

6 SEMAPHORES (Semaforlar)

🔑 Temel

- ❑ **Semafor (S):** Bir tamsayıdır (integer), sadece **2 atomik işlemle** erişilir:
 - 🔴 `wait()` — literatürde `P()`
 - 🔴 `signal()` — literatürde `V()`

💻 Basit (busy-wait'li) tanım:

```
wait(S) {                                signal(S) {
    while (S <= 0)                        S++;
        ; // busy wait                    }
    S--;
}
```

- 🔴 `wait()` ve `signal()` **atomik (kesintisiz)** çalışır → bir process S'yi değiştirirken başkası değiştiremez.
- `wait()` içindeki **$S \leq 0$ testi** de interrupt olmadan yapılmalı.

📊 2 Semafor Türü (TANIM SORUSU!)

Tür	Değer Aralığı	Kullanım
❑ Counting semaphore	Kısıtsız (herhangi bir tamsayı)	Belirli sayıdaki kaynağa erişimi denetler; kaynak sayısı yla başlatılır
❑ Binary semaphore	0 veya 1	Mutex lock gibi davranır

📝 BOŞLUK DOLDURMA

- `wait()` literatürde (P), `signal()` (V) ile gösterilir.
- Değeri 0 veya 1 olan semafor (**binary semaphore**); kaynak sayısıyla başlatılan (**counting semaphore**).

🔗 İşlem Bağımlılığı (Synchronization) Örneği

? **Slayttaki klasik soru:** P2'deki S2 komutu, P1'deki S1'den **SONRA** çalışmak zorunda. Ortak `synch` semaforu ile nasıl?

● CEVAP:

- `synch` başlangıç değeri = 0

```
// P1                // P2
S1;                  wait(synch); // synch>0 olana kadar bekler
signal(synch);       S2;
// synch artırılır
```

- P1, S1'i çalıştırıp `signal(synch)` ile `synch`'i artırır.
- P2, `wait(synch)` ile `synch>0` olana kadar **bekler** → S1 garanti S2'den önce çalışır.

📝 BOŞLUK DOLDURMA

- S2'nin S1'den sonra çalışması için `synch` başlangıç değeri (0) olmalı.

🔧 Semaphore Gerçekleştirimi (Busy-Waiting'siz, Doğru Versiyon)

- Busy waiting yerine process kendini **bloke (block)** eder:

- `block()` → process'i **waiting state**'e alır, semaforun **bekleme kuyruğuna** koyar.
- CPU scheduler başka process seçer.
- `signal()` gelince `wakeup()` → process'i **waiting** → **ready state**'e geçirir, **ready queue**'ya koyar.

📁 Veri yapısı ve işlemler:

```
typedef struct {
    int value;
    struct process *list;    //  bekleyen process listesi
} semaphore;

wait(semaphore *S) {
    S->value--;
    if (S->value < 0) {
        add this process to S->list;
        block();
    }
}

signal(semaphore *S) {
    S->value++;
    if (S->value <= 0) {
        remove a process P from S->list;
        wakeup(P);
    }
}
```

-  **NEGATİF değer = bekleyen process sayısını gösterir!** (önemli soru)

BOŞLUK DOLDURMA

- `block()` process'i (beklemeye/waiting state'e) alır, `wakeup()` (çalışmaya devam ettirir/ready state'e geçirir).
- Semaforun negatif değeri (bekleyen process sayısını) gösterir.

DEADLOCK (Ölümcül Kilitlenme)

- ☐ **Deadlock:** İki veya daha fazla process'in, devam edebilmek için **birbirinin bitirmesini beklediği** ve hiçbirinin devam edemediği durum.
-  **İki veya daha fazla process'in SONSUZA KADAR birbirini beklemesi.**

 **Slayttaki örnek (S ve Q semaforları, başlangıç=1):**


```
// P0          // P1
wait(S);       wait(Q);
wait(Q);        wait(S);  ÇAPRAZ!
...           ...
signal(S);     signal(Q);
signal(Q);     signal(S);
```

 Po → `wait(S)` (S=0), P1 → `wait(Q)` (Q=0). Sonra Po `wait(Q)` ister (P1'i bekler), P1 `wait(S)` ister (Po'ı bekler) → **DEADLOCK!**

STARVATION (Açlık) ve PRIORITY INVERSION

Starvation

- ❑ **Starvation / indefinite blocking:** Process'in semaforda **sonsuz** kadar beklediği durum.
- Örn. bekleme listesinden **LIFO (son giren ilk çıkar)** sırayla çıkarılırsa starvation olur.
- ⚠ **Deadlock vs Starvation farkı:**
 - **Deadlock:** Hiçbir process ilerleyemez (karşılıklı bekleme).
 - **Starvation:** Yüksek öncelikliler kaynağı sürekli kapar, **düşük öncelikli** belirsizce engellenir (ama sistem ilerler).

Priority Inversion

- ❑ **Priority inversion:** Yüksek öncelikli process (H), düşük öncelikli (L) tarafından tutulan kaynağı beklemek zorunda kalır. Araya orta öncelikli (M) girerse durum karmaşıklaşır. (Öncelik: $L < M < H$)
- **Çözüm: priority-inheritance protocol** → Düşük öncelikli process, yüksek önceliklinin ihtiyacı olan kaynağı tutuyorsa, **geçici olarak yüksek önceliği devralır**; işi bitince eski önceliğine döner.

BOŞLUK DOLDURMA

- İki process'in sonsuza kadar birbirini beklemesi (**deadlock**).
- Process'in semaforda süresiz beklemesi (**starvation / indefinite blocking**).
- Priority inversion çözümü (**priority-inheritance protocol**).

7 CLASSICAL PROBLEMS OF SYNCHRONIZATION

● **3 klasik problem** (yeni senkronizasyon yöntemlerini test etmek için kullanılır), çözümlerinde **semaforlar** kullanılır:

1. ☐ **Dining-Philosophers Problem** (Yemek Yiyen Filozoflar)
2. ☐ **Bounded-Buffer Problem** (= Producer-Consumer)
3. ☐ **Readers and Writers Problem**

7.1 Dining-Philosophers Problem

- **5 filozof**, masada **5 çubuk**, ortada pirinç. Filozof yemek için **2 çubuğa** ihtiyaç duyar (sol + sağ).
- Bir filozof bir seferde **1 çubuk** alır; yedikten sonra **ikisini bırakır**.
- ● Bu problem, kaynakların **deadlock ve starvation olmadan** tahsis edilmesinin temsidir.

Semafor Çözümü

```
semaphore chopstick[5]; // ● hepsinin başlangıç değeri = 1
while (true) {
    wait(chopstick[i]);           // sol çubuk
    wait(chopstick[(i+1) % 5]);   // sağ çubuk
    /* eat for awhile */
    signal(chopstick[i]);         // sol bırak
    signal(chopstick[(i+1) % 5]); // sağ bırak
    /* think for awhile */
}
```

-  `chopstick[]` dizisinin başlangıç değeri (**1**, her eleman).
- `wait` → çubuğu alır (değer 1→0), `signal` → çubuğu bırakır (değer 0→1).

DEADLOCK PROBLEMİ:

● Bu çözüm *mutual exclusion* sağlar (iki komşu aynı anda yemez), AMA **DEADLOCK yaratabilir!** 5 filozof da aynı anda acıkıp sol çubuğunu tutarsa → tüm `chopstick[]=0` → herkes sağ çubuğu bekler → **sonsuz kadar bekleme**.

● DEADLOCK Çözümleri (3 yöntem):

1. En fazla 4 filozof aynı anda masaya otursun.

2. Filozof çubukları **yalnızca ikisi de müsaitse** alsın (kritik bölümde alma).
3. ☐ **Asimetrik çözüm: Tek** sayılı filozof önce **sol** sonra sağ; **çift** sayılı filozof önce **sağ** sonra sol alır.

İŞLEM SORULARI (slayttan):

- ? "5 filozof aynı anda `wait(chopstick[i])` çalıştırınca kaç filozof yer?" → **Hiçbiri (deadlock)** / **0** (eğer hepsi sol çubuğu tutar ve sağ beklerse).
- ? "P₀ ve P₂ aynı anda yemek isterse aynı anda kaç filozof yer?" → **2** (P₀ ve P₂ komşu değil, çubukları çakışmaz).
- ? "Hangi çiftler aynı anda yiyebilir?" → **Komşu OLMAYAN** çiftler (örn. P₀-P₂, P₁-P₃, P₂-P₄...). **Komşular** (P₀-P₁ gibi) aynı anda **yiyeemez**.

7.2 Bounded-Buffer (Producer-Consumer) Problem

3 semafor/değişken (başlangıç değerleriyle EZBERLE!):

```
int n; // buffer boyutu

semaphore mutex = 1; // ☐ BINARY - buffer'a erişimde mutual exclusion
semaphore full = 0; // ☐ COUNTING - dolu buffer sayısı
semaphore empty = n; // ☐ COUNTING - boş buffer sayısı
```

BOŞLUK DOLDURMA (çok kritik!)

Semafor	Başlangıç	Tür	Görev
<code>mutex</code>	1	Binary	buffer'a erişimde mutual exclusion
<code>full</code>	0	Counting	dolu buffer sayısı
<code>empty</code>	n	Counting	boş buffer sayısı

Producer

```
do {  
    /* eleman üret */  
    wait(empty);    //  boş yer yoksa bekle  
    wait(mutex);    //  mutual exclusion  
    buffer[in] = next_produced;  
    in = (in+1) % BUFFER_SIZE;  
    counter++;  
    signal(mutex);  //  kilidi bırak  
    signal(full);    //  dolu sayısını artır  
} while (true);
```

Consumer

```
do {  
    wait(full);      //  dolu eleman yoksa bekle  
    wait(mutex);    //  mutual exclusion  
    next_consumed = buffer[out];  
    out = (out+1) % BUFFER_SIZE;  
    counter--;  
    signal(mutex);  //  kilidi bırak  
    signal(empty);  //  boş sayısını artır  
    /* eleman tüket */  
} while (true);
```

 **Sıra önemli!** Önce `wait(empty)/wait(full)` SONRA `wait(mutex)`. Tersî olursa **deadlock** olur.

 **Slayt sorusu:** "empty ve full yerine sadece tek bir `count` semaforu olsaydı düzgün çalışır mıydı?" → **HAYIR**, çünkü hem dolu hem boş durumu ayrı ayrı kontrol edilemez, senkronizasyon bozulur.

7.3 Readers-Writers Problem

- ☐ **Readers:** Veriyi sadece **okur**, güncellemez.
- ☐ **Writers:** Veriyi **günceller**.
-  **Kural tablosu:**

Durumdaki	Gelmek isteyen	Sonuç
W	W	✗ no W
W	R	✗ no R
R	W	✗ no W
R	R	✓ R (izin var!)

● **2 reader aynı anda okuyabilir** (sorun yok). Ama **1 writer + başka process (R ya da W) aynı anda erişirse kaos olur.**

🔗 2 Varyasyon (TANIM SORUSU!)

Varyasyon	Kural	Kim aç kalır?
☐ First (Reader önceliği)	Writer izin almadıkça hiçbir reader bekletilmez	Writer'lar starve olur
☐ Second (Writer önceliği)	Writer hazırsa, yeni reader okumaya başlayamaz	Reader'lar starve olur

💻 Çözüm (First variation) – Paylaşılan Veriler

```
semaphore rw_mutex = 1; // ● writer-reader arası mutual exclusion
semaphore mutex = 1;    // ● read_count güncellemesinde mutual exclusion
int read_count = 0;      // ● o an okuyan reader sayısı
```

💻 Writer Process

```
while (true) {
    wait(rw_mutex);
    /* writing is performed */
    signal(rw_mutex);
}
```

- Aynı anda birden çok writer CS'de olamaz → `rw_mutex` ile sağlanır.

📖 Reader Process

```
while (true) {
    wait(mutex);
    read_count++;
    if (read_count == 1)          // 🏠 İlk reader
        wait(rw_mutex);          //   writer'ı kilitler
    signal(mutex);
    /* reading is performed */
    wait(mutex);
    read_count--;
    if (read_count == 0)          // 🏠 SON reader
        signal(rw_mutex);         //   writer'a izin ver
    signal(mutex);
}
```

🔴 Önemli Soru-Cevaplar (slayttan):

- **rw_mutex ne işe yarar?** → Writer ile başka process (R/W) arasında mutual exclusion. **İlk giren** ve **son çıkan** reader tarafından kullanılır.
- **mutex ne işe yarar?** → **read_count** güncellenirken mutual exclusion sağlar. Olmasaydı → **read_count**'ta **race condition** olurdu.
- **if (read_count == 1) ne sağlar?** → **İlk reader** girerken writer'ı kilitler (**wait(rw_mutex)**). Olmasaydı → her reader **rw_mutex**'i alır, **birden fazla reader giremezdi / kilitlenirdi**.
- **if (read_count == 0) ne sağlar?** → **Son reader** çıkınca writer'a izin verir (**signal(rw_mutex)**). Olmasaydı → son reader **rw_mutex**'i bırakmaz, **writer hiç giremez (deadlock/starvation)**.

📊 İŞLEM (state takibi – slayttaki tablo):

Başlangıç: **read_count=0, rw_mutex=1, mutex=1**

Olay	read_count	rw_mutex	mutex
Başlangıç	0	1	1
Writer W1 girer	0	0	1
W1'deyken Reader gelir, wait(mutex)	0	0	0
read_count++ → 1, wait(rw_mutex) 'te bloke	1	0	0
→ Reader writer bitene kadar bekler ✅			

✅ Bu çözümle: *birden fazla reader aynı anda okuyabilir; writer varken kimse giremez.*

BOŞLUK DOLDURMA

- `rw_mutex` ve `mutex` başlangıç (1), `read_count` (0).
- İlk reader writer'ı (**kilitler** / **wait(rwmutex)**), son reader _ (**serbest bırakır** / **signal(rw_mutex)**).
- Bazı sistemlerde problem (**reader-writer locks**) sağlayan kernel ile çözülür.

⚡ SINAV ÖNCESİ HIZLI TEKRAR KARTLARI

🎯 En Çok Sorulan Tanımlar

Terim	Tek Cümle
Race condition	Sonucun erişim sırasına bağlı olduğu durum
Critical section	Paylaşılan veriye erişen kod bölümü
Mutual exclusion	Aynı anda tek process CS'de
Progress	CS boşken seçim süresiz ertelenemez
Bounded waiting	Beklemeye limit var
Mutex/Spinlock	En basit araç, busy-wait dezavantajı
Binary semaphore	0/1, mutex gibi
Counting semaphore	Kaynak sayısı kadar, kısıtsız
Deadlock	Karşılıklı sonsuz bekleme
Starvation	Süresiz engellenme
Priority inversion	Çözümü: priority-inheritance

1 2 3 4 Ezberlenecek Başlangıç Değerleri

- Producer-Consumer: `mutex=1`, `full=0`, `empty=n`
- Readers-Writers: `rw_mutex=1`, `mutex=1`, `read_count=0`
- Dining Philosophers: `chopstick[5]` hepsi `=1`
- İşlem bağımlılığı (synch): `synch=0`
- testandset lock: `lock=false`

⚠ En Çok Karıştırılanlar

- **Peterson** → yazılım, **2 process**, modern mimaride **garanti DEĞİL** (reordering).

- **testandset** → her zaman `true` yapar, eski değeri döner.
- **compareandswap** → her zaman orijinal değeri döner, eşitse swap.
- **wait()=P(), signal()=V()**.
- **Semafor negatif değeri = bekleyen process sayısı.**
- Producer-Consumer'da sıra: önce **wait(empty/full)**, sonra **wait(mutex)**.
- Dining: aynı anda yiyebilenler = **komşu OLMAYANLAR**.
- Readers: `read_count==1` → ilk reader writer'ı kilitler; `==0` → son reader serbest bırakır.

🍀 **Başarılar! Slaytlardan kelimesi kelimesine boşluk doldurma gelirse hazırsın.**

Hoca özellikle: 3 gereksinim (ME/Progress/Bounded), semafor başlangıç değerleri, testandset/compareandswap mantığı, deadlock örnekleri ve Readers-Writers if kontrollerinden soruyor.